

Perceptrón + Ingeniería de Características

Resolviendo la compuerta XOR con un kernel polinomial

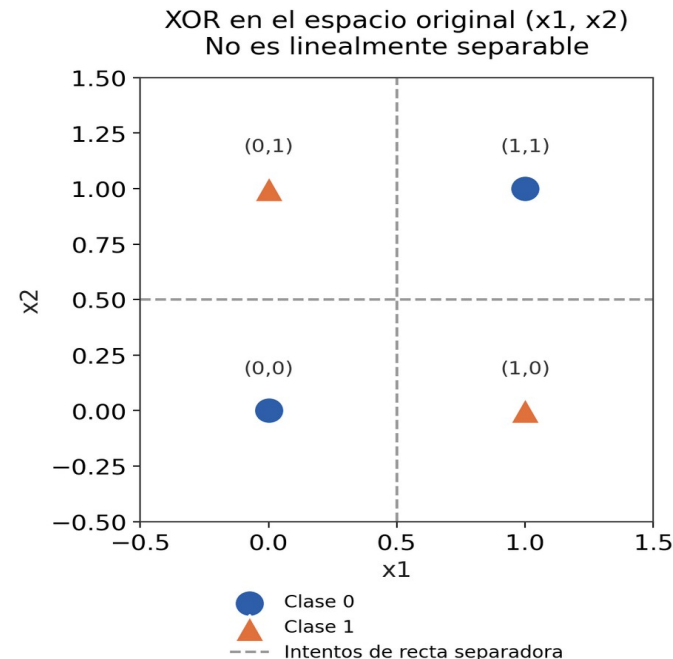
El problema: XOR no es linealmente separable

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Un perceptrón simple traza una única recta (hiperplano) en el plano (x1, x2).

Ningún hiperplano puede separar $\{(0,0),(1,1)\}$ de $\{(0,1),(1,0)\}$: los puntos de cada clase quedan en esquinas opuestas del cuadrado.

Solución: ingeniería de características — proyectar los datos a un espacio donde SÍ sean linealmente separables.



Como se observa, las clases 0 (azul) y 1 (naranja) están entrelazadas: cualquier recta que se intente deja siempre un punto mal clasificado.

Kernel utilizado: polinomial de grado 2

Mapeo de características (feature map) explícito:

$$\phi(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_1 \cdot \mathbf{x}_2)$$

¿Por qué este kernel?

El kernel polinomial completo $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + 1)^2$ induce, entre otros, el término cruzado $\mathbf{x}_1 \cdot \mathbf{x}_2$. Ese término es exactamente lo que necesitamos: codifica la interacción entre las dos entradas, que es la propiedad que define al XOR.

En vez de operar en un espacio kernelizado implícito, se usa la versión explícita $\phi(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_1 \mathbf{x}_2)$, agregando $\mathbf{x}_1 \mathbf{x}_2$ como una tercera característica de entrada al perceptrón.

Transformación de los 4 puntos de XOR

$(0,0) \rightarrow (0, 0, 0)$ clase 0

$(0,1) \rightarrow (0, 1, 0)$ clase 1

$(1,0) \rightarrow (1, 0, 0)$ clase 1

$(1,1) \rightarrow (1, 1, 1)$ clase 0

Ahora la clase 0 y la clase 1 ocupan regiones separables por

Entrenamiento del perceptrón

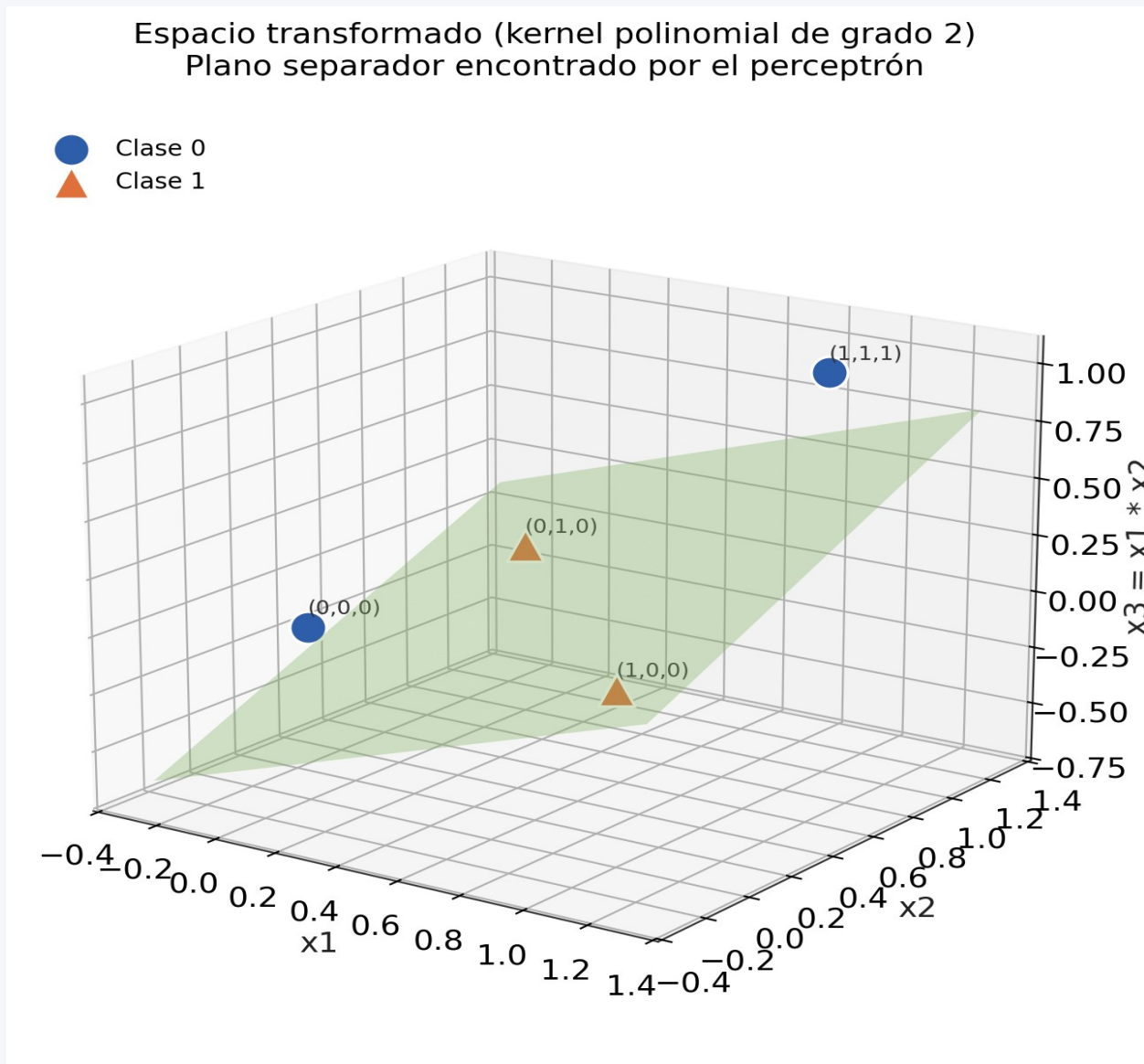
Algoritmo (regla de aprendizaje del perceptrón)

1. Inicializar pesos $w = (w_1, w_2, w_3)$ y sesgo b al azar.
2. Para cada ejemplo $\phi(x_1, x_2)$, calcular $z = w \cdot \phi(x) + b$.
3. Predecir $\hat{y} = 1$ si $z \geq 0$, si no $\hat{y} = 0$ (función escalón).
4. Actualizar: $w \leftarrow w + \eta(y - \hat{y})\phi(x)$, $b \leftarrow b + \eta(y - \hat{y})$.
5. Repetir hasta que no haya errores (convergencia).

Implementado en JavaScript (Node.js) — clase
Perceptron con $\text{featureMap}(x_1, x_2) = [x_1, x_2, x_1 \cdot x_2]$
aplicado antes de entrenar.

```
function featureMap([x1, x2]) {  
    return [x1, x2, x1 * x2];  
}  
  
train(X, y, epochs) {  
    for (epoch of epochs) {  
        for (i in X) {  
            pred = predict(X[i])  
            err = y[i] - pred  
            w += lr * err * X[i]  
            b += lr * err  
        }  
    }  
}
```

Resultado: el plano separador en 3D



Pesos aprendidos por el perceptrón

$$w1 = 0.119 \quad w2 = 0.128 \quad w3 = -0.272 \quad b = -0.100$$

Ecuación del plano separador:

$$0.119 \cdot x1 + 0.128 \cdot x2 - 0.272 \cdot (x1x2) - 0.100 = 0$$

Convergencia: 8 épocas, tasa de aprendizaje $\eta = 0.1$.

El plano (verde) deja a un lado los puntos (0,0,0) y (1,1,1) —clase 0— y al otro lado (0,1,0) y (1,0,0) —

Verificación de resultados

(x_1, x_2)	$\phi(x_1, x_2, x_1x_2)$	XOR esperado	Predicción	¿Correcto?
(0, 0)	(0, 0, 0)	0	0	✓
(0, 1)	(0, 1, 0)	1	1	✓
(1, 0)	(1, 0, 0)	1	1	✓
(1, 1)	(1, 1, 1)	0	0	✓

El perceptrón, entrenado sobre las características expandidas $\phi(x_1, x_2) = (x_1, x_2, x_1x_2)$, clasifica correctamente las 4 combinaciones de XOR: 100% de exactitud, algo imposible de lograr con el perceptrón en el espacio original (x_1, x_2) .

Conclusiones

- XOR no es linealmente separable en el espacio original de entradas (x_1, x_2) .
- La ingeniería de características con el kernel polinomial $\phi(x_1, x_2) = (x_1, x_2, x_1x_2)$ proyecta los datos a un espacio 3D donde sí existe un hiperplano (plano) separador.
- Un perceptrón simple, entrenado sobre esas características expandidas, converge en pocas épocas y clasifica correctamente el 100% de los casos.
- La técnica generaliza la idea del "kernel trick": en vez de cambiar el clasificador, se cambia la representación de los datos.